

EVALUACIÓN FINAL

Aventura en el Reino Oscuro

Alumno: Federico Fabián Silva
Materia: Paradigmas de Programación
Año: 2026

1. Introducción

El proyecto consiste en un juego de aventura por consola desarrollado en Python. El objetivo fue aplicar los temas vistos durante la cursada dentro de un programa completo, con un menú, creación de personaje, combates, niveles, tienda y guardado de partida.

2. Objetivo y reglas del juego

El jugador debe recorrer cuatro zonas y derrotar al jefe de cada una. Para llegar al jefe necesita vencer cierta cantidad de enemigos. Durante la partida obtiene experiencia y monedas, puede comprar pociones o mejoras, descansar y guardar su avance. La partida termina cuando el personaje pierde toda la vida o derrota al jefe final.

3. Organización del programa

El código está dividido en funciones para que cada parte tenga una responsabilidad concreta. Por ejemplo, hay funciones para validar opciones, crear enemigos, combatir, comprar objetos, guardar la partida y mostrar los menús. Esta separación hace que el programa sea más fácil de leer y corregir.

4. Conceptos de Python aplicados

Variables

Se utilizan para guardar datos temporales y permanentes, como el nombre del jugador, la vida, el oro, el nivel y la opción elegida en cada menú.

Condicionales

Se usan if, elif y else para tomar decisiones. Por ejemplo, permiten elegir una clase, comprobar si hay suficiente oro y decidir qué acción ejecutar en combate.

Ciclo while

Mantiene activos el menú principal, el menú del juego y los combates. También se usa para repetir una pregunta cuando el usuario escribe una opción incorrecta.

Ciclo for

Python lo utiliza dentro de algunas operaciones del programa y se incluyó el manejo de colecciones mediante listas de enemigos. La selección concreta se realiza con random.choice.

Funciones

Cada función resuelve una tarea: crear_jugador, combate, tienda, guardar_partida, cargar_partida, entre otras.

Listas y diccionarios

Las listas contienen enemigos posibles. Los diccionarios guardan los datos del jugador, los enemigos y los jefes mediante claves como vida, ataque y defensa.

Archivos

La partida se guarda en formato JSON. Esto permite cerrar el juego y continuar más adelante con los mismos datos.

Números aleatorios

El módulo random se usa para elegir enemigos, generar eventos, variar el daño y calcular golpes críticos o intentos de escape.

5. Explicación de las funciones principales

`pedir_opcion()`

Recibe un mensaje y una lista de opciones válidas. Usa un `while` para seguir preguntando hasta obtener una respuesta correcta.

`crear_jugador()`

Solicita el nombre y la clase. Según la clase elegida asigna estadísticas diferentes y devuelve un diccionario con todos los datos.

`combate()`

Es la parte principal del juego. Mientras ambos personajes tengan vida, muestra acciones y calcula el turno del jugador y del enemigo.

`calcular_daño()`

Resta la defensa al ataque y agrega una variación aleatoria. Si el resultado es menor que uno, se fuerza un daño mínimo.

`subir_nivel()`

Compara la experiencia acumulada con la necesaria. Al subir de nivel aumenta vida, ataque, defensa y energía.

`tienda()`

Permite comprar pociones o mejoras. Antes de cada compra verifica si el jugador posee suficiente oro.

`guardar_partida()` y `cargar_partida()`

Transforman los datos del jugador a JSON y los recuperan. También incluyen manejo de errores por si el archivo no existe o está dañado.

`jugar()`

Contiene el menú de la partida y conecta las demás funciones. El ciclo finaliza si el jugador vuelve al menú, pierde o gana el juego.

6. Ejemplo del flujo de una partida

Primero se crea el personaje. Después el programa entra al menú de juego. Al elegir Explorar puede aparecer un enemigo o un evento. Si aparece un enemigo comienza el combate. Una victoria entrega experiencia y oro. Cuando se cumplen los requisitos se puede enfrentar al jefe, avanzar de zona y continuar hasta el final.

7. Validaciones realizadas

Las opciones de los menús se validan para impedir valores no permitidos. También se verifica que el nombre no esté vacío, que el jugador tenga pociones, energía u oro suficiente y que exista un archivo antes de cargar una partida. Estas validaciones evitan varios errores durante la ejecución.

8. Pruebas realizadas

Se comprobó la creación de las tres clases, las opciones de los menús, el ataque normal, las habilidades, el uso de pociones, la compra de mejoras, el aumento de nivel y el guardado y carga. También se probaron entradas incorrectas para verificar que el programa vuelva a pedir una opción.

9. Conclusión

El trabajo permitió integrar los conceptos básicos de Python en un único programa. La parte más compleja fue coordinar el combate con las estadísticas y lograr que los cambios del jugador se conservaran durante toda la partida. Dividir el código en funciones facilitó el desarrollo y las pruebas.

10. Enlaces

Repositorio de GitHub: <https://github.com/Federic10/aventura-reino-oscuro.git>

Video explicativo:

https://drive.google.com/drive/folders/1_o1LWUOqFkI1lXTKd9ApU9uxRfX3apUC?usp=drive_link

11. Archivos incluidos en la entrega

- aventura_reino_oscuro.py
- Informe_Final_Federico_Silva.docx
- Informe_Final_Federico_Silva.pdf
- README.md